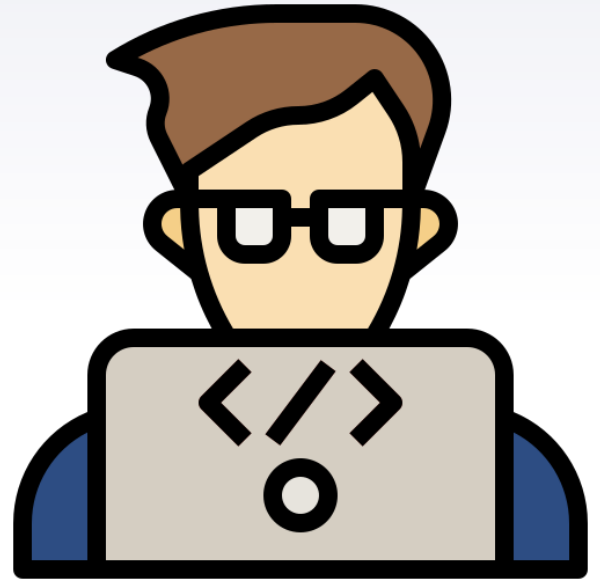


OBJECT ORIENTED PROGRAMMING in C++



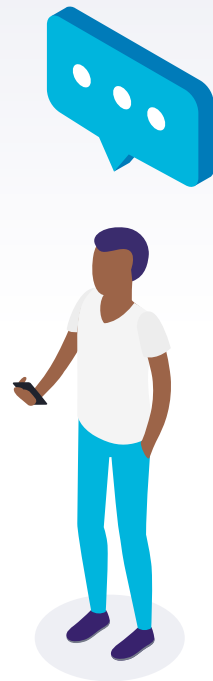
Bucharest University of Economic Studies,
Faculty of Cybernetics, Statistics and Economic Informatics

HELLO!

I am Prof. Catalin Boja

Coder & Lecturer for 20+ years

You can find me at catalin.boja@ie.ase.ro





The purpose of this document is to structure and present briefly the concepts discussed during the course. Preparing for the exam, based solely on this material represents a superficial approach



Objective

To teach how to think and code in Object Oriented (OO) paradigm.....and

- **to code in C++** (write source code, compile it, debug it and run it)
- **to design, understand, verify, debug, modify and defend a C++ solution—** including one produced with AI



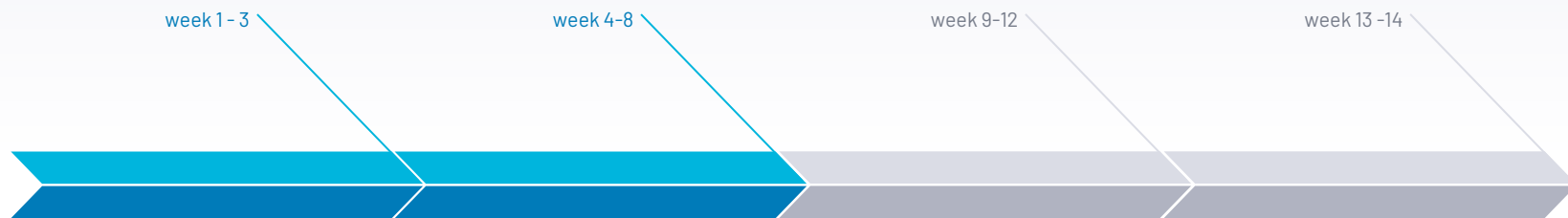
Evaluation

- ▶ 50% seminar and course activities
 - ▶ 15% - 1 practical test (with no compiler errors) – after first 4-5 weeks
 - ▶ 15% - 1 practical test (with no compiler errors) – after first 8-9 weeks
 - ▶ 10% - quiz test (concepts and sample code questions) – 14th week
 - ▶ 5% - course quizzes and live questions
 - ▶ 5% - small assignments/tests/quizzes at seminar – be active during the semester
- ▶ 50% **final oral exam** – practical exam that require the writing of a free compiler error C++ application that works and explaining the OOP concepts
- ▶ you need 1 points out of 5 and minimum 4 attendances to take the exam in January/February
- ▶ you need at least 5 in the exam

Contents

- ▶ Review of necessary background
- ▶ Classes (definition, attributes, constructors, methods, etc.)
- ▶ Overloading operator methods
- ▶ Streams (standard and files)
- ▶ Inheritance (Hierarchies of classes , Polymorphism, Virtual methods)
- ▶ STL – Standard Template Library

Our process is easy



Recap

Pointers, Functions,
Dynamic and Static
Arrays, String

Basics

Classes definition,
Overloading,
Operators, Streams,
Files

Advanced

Inheritance, Composition
Overriding, Has-A vs. Is-A,
Virtual, Abstract

More

STL, Collections

Resources

- ▶ www.acs.ase.ro/cpp-en
- ▶ The standard: Bjarne Stroustrup – The Creator of C++, “The C++ Programming Language”-3rd Edition, Addison-Wesley
- ▶ <https://en.cppreference.com/w/>
- ▶ Any tutorial found on Internet/YouTube
- ▶ Any help you get from an AI assistant



1

What, Why & How

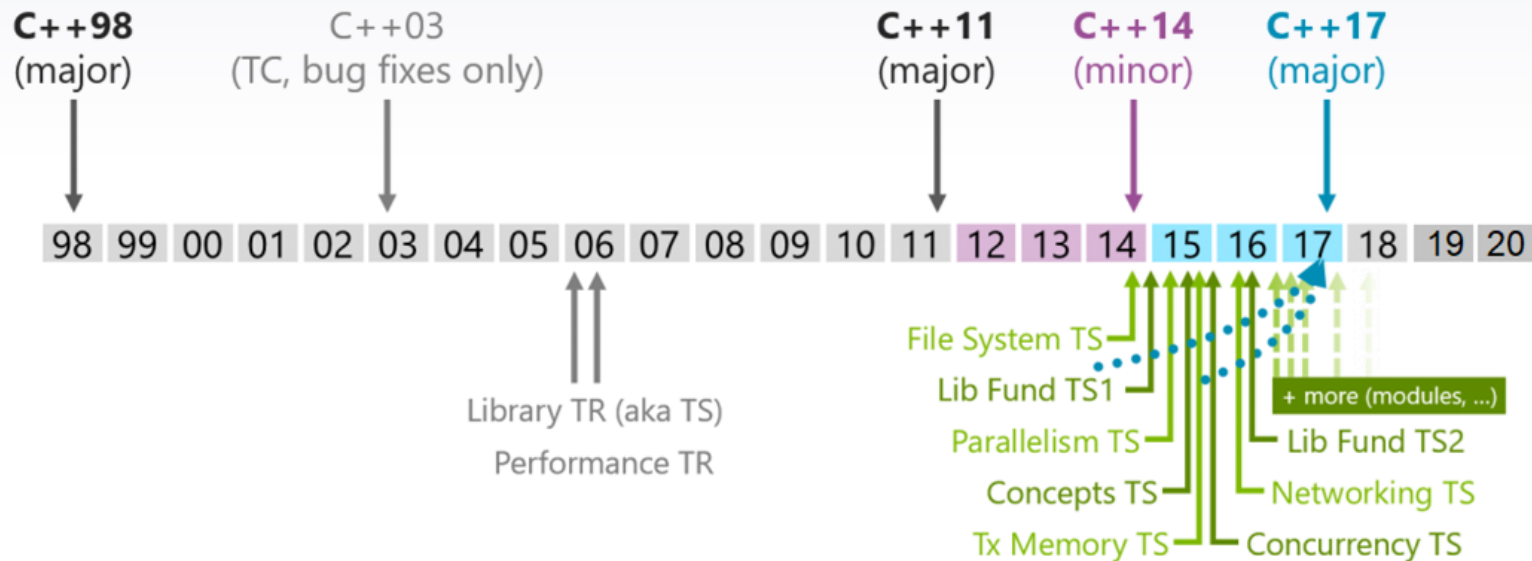
Why learning OOP in C++ and How ?



► What is C++

- ▶ The C++ language is an Object-Oriented Programming (OOP) language
- ▶ Created, designed & developed by Bjarne Stroustrup in 1979 as a C language upgrade
- ▶ Initially known as “C with classes”, it was renamed C++ in 1983. The ++ stands for incrementing, hence is an incremented or evolved C
- ▶ Widely used for application development for different platforms

What is C++



Source: <https://isocpp.org/std/status>

▶ What is C++

- ▶ Very fast
- ▶ Provides specific implementation for a lot of hardware
- ▶ No need for a virtual machine or an interpreter
- ▶ For many years in top 5 used programming languages in the TIOBE index (2nd in September 2025) - <https://www.tiobe.com/tiobe-index/>

What is C++

 C++	 JavaScript
 Java/C#	 PHP(Without MySQL)
 Ruby	 Pascal
 Perl	 Lisp
 Visual Basic	 Haskell
 Python	 C

► What is OOP ?

- **Object-Oriented Programming (OOP)** is a programming paradigm that organizes software design around objects, which are instances of **classes**. OOP focuses on using real-world entities to represent concepts in software, promoting modularity, reusability, and a structured way to manage complexity in code.

► Why OOP ?

Because is all about
abstraction

Managing Complexity Through Abstraction
Taking real life story and translate it into code

► Why OOP ?

- ▶ You will be able to design software solutions for real life problems – abstraction
- ▶ 4 magic words: Encapsulation, Composition, Inheritance, Polymorphism
- ▶ Concepts are present in all Object Oriented Languages (.NET platform – C#, Java, Kotlin, C++, Python, Objective-C) or Object Based Languages (JavaScript)

► Why OOP ?

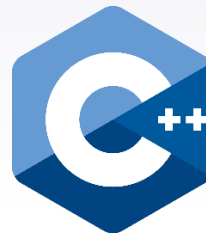
State — Behavior — Responsibility —
Lifetime — Relationships

+

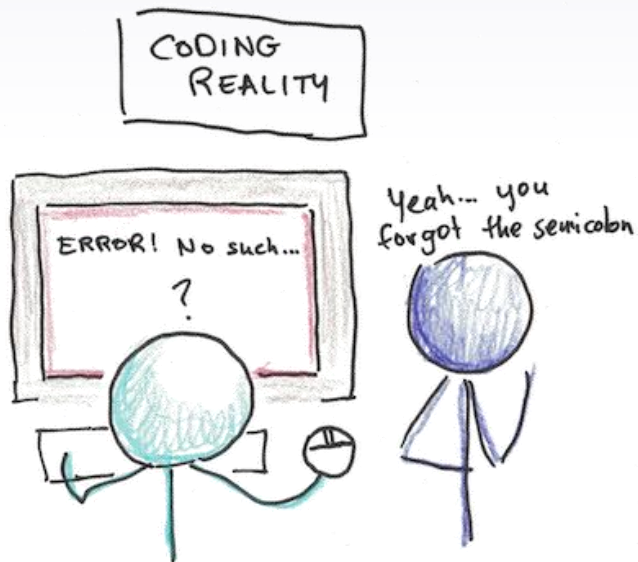
Encapsulation — Composition —
Inheritance — Polymorphism

► Why OOP ?

- ▶ Objects
- ▶ Classes
- ▶ Inheritance
- ▶ Encapsulation
- ▶ Frameworks of classes
- ▶ Abstraction



► How to learn to code



Source: Scott H. Young

<https://www.scotthyoung.com/blog/2019/07/08/learn-to-code/>

► How to learn to code

- ▶ Learning how to code is based on numerous failures
 - ▶ you need to try to solve different problems using C++ by yourself (after you think you got the concepts) and this will lead to a lot of failures
 - ▶ solving all the problems (even if takes time.....sometimes too much) will help you learn

► How to learn to code

- ▶ You need to
 - ▶ Accept that there is always room for improvement (nobody is perfect)
 - ▶ Be persistent in achieving your goals
 - ▶ Use your imagination to find solutions

▶ How to learn to code

- ▶ Don't cheat
 - ▶ Don't copy & paste programs
 - ▶ Don't ask a bot or another person to do your assignments and projects
 - ▶ Don't 'vibe' code (with the AI) the assignments

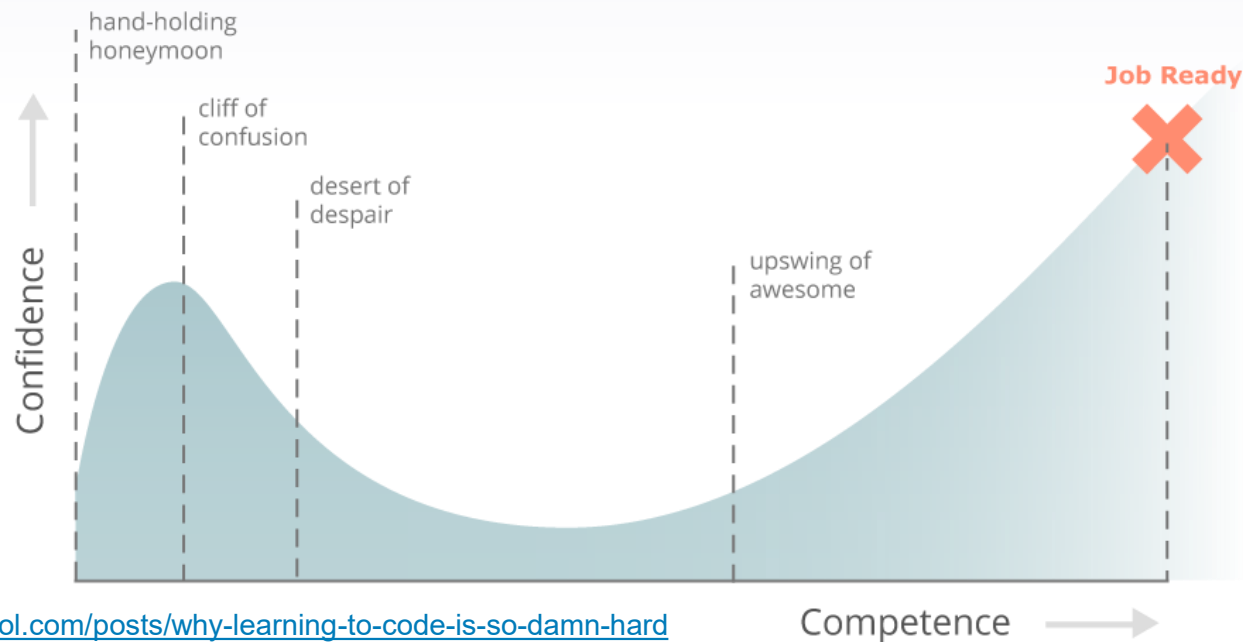


We learn wisdom from failure much more than from success. We often discover what will do, by finding out what will not do; and probably he who never made a mistake never made a discovery"

Samuel Smiles, The Lives Of George And Robert Stephenson

How to learn to code

Coding Confidence vs Competence



“

OUR LIFE

IS THE SUM TOTAL OF
ALL THE DECISIONS WE
MAKE EVERY DAY,
AND THOSE
DECISIONS ARE
DETERMINED BY OUR PRIORITIES.

- MYLES MUNROE

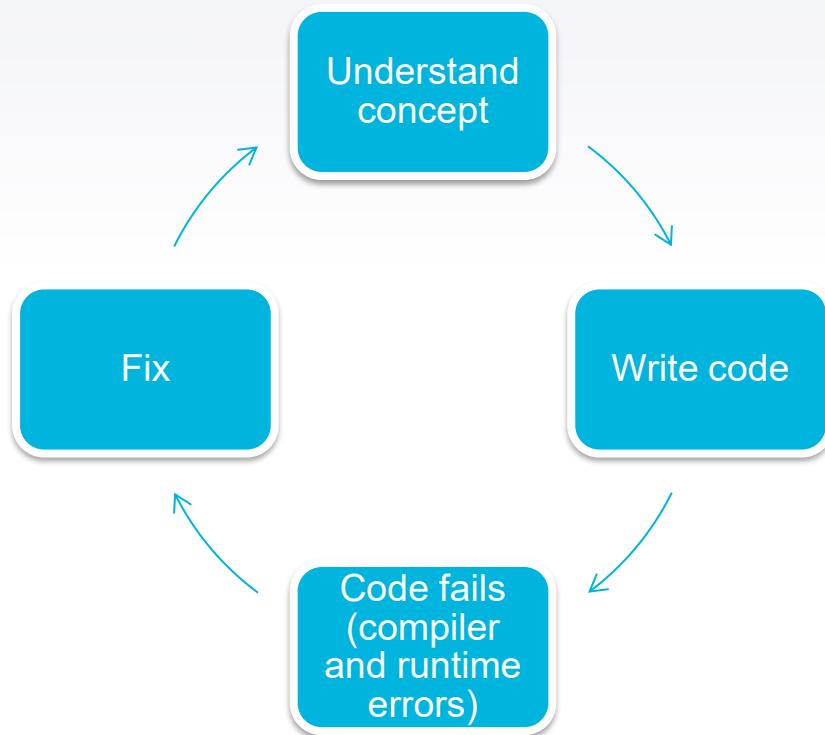


Center for
Management &
Organization
Effectiveness

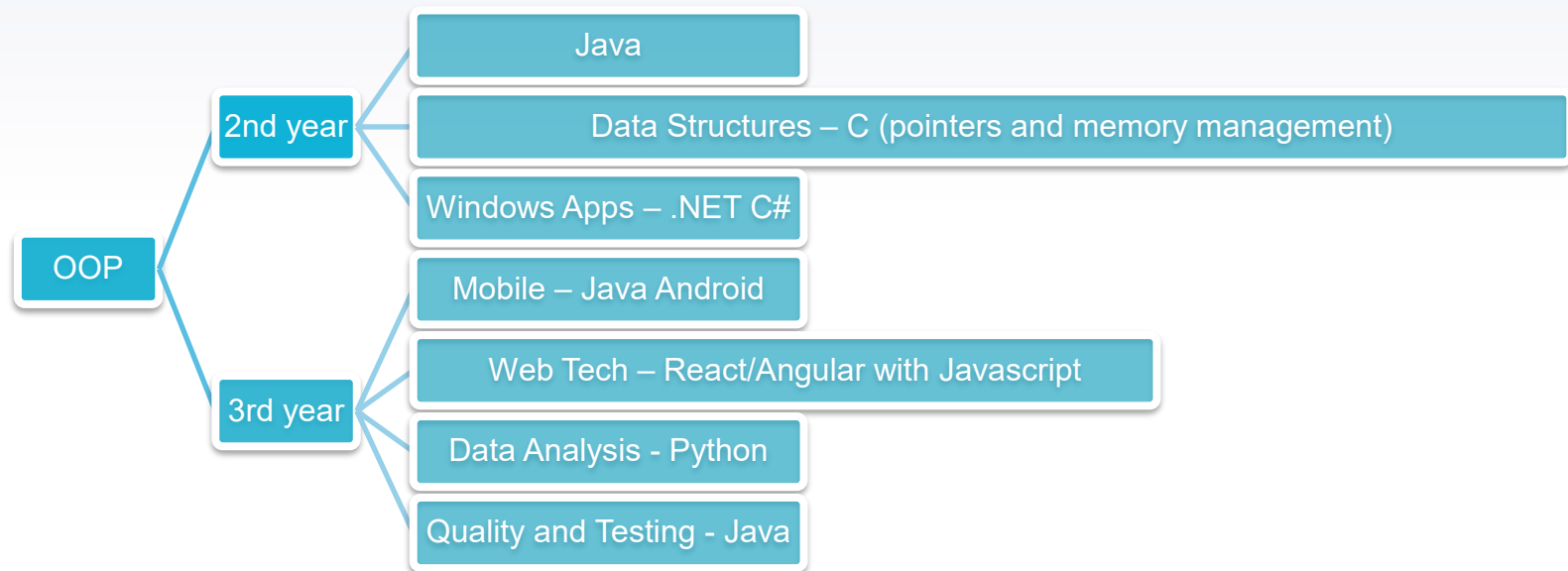
► How to learn to code

1. After each course and seminar, check if you understood the concepts (otherwise read and ask colleagues and teachers)
2. Write as many C ++ programs as possible (if all are ok from the first attempt this is not okay) ... but do not copy them from the course and seminar examples (define your own examples)
3. If all the initial attempts have compilation and / or execution errors you are on the right track
4. If you are in step 2, do not give up fix them by reading the course support and looking for resources on the Internet (not everything you find on stackoverflow is right)
5. If you DO NOT succeed in 2-3 hours to correct a problem ask for help (colleagues, teachers, AI)
6. DO NOT give up writing as many programs as possible ... you need to code 2-3 hours a week
7. Use AI to generate exercises, to evaluate your solutions and to give you information

► How to learn to code



Bonus



2

Prerequisites

What to remember, Tools, Recap



► What to remember

1. Data types (possible values and size)
2. Hexadecimal and binary representation
3. Functions/Methods/Subprograms (definition and calling)
4. At least one sorting algorithm
5. At least one searching algorithm
6. Using VS 2022 (editing/compiling/running and debugging)

Tools

- ▶ Microsoft Visual Studio 2026
- ▶ [Download Visual Studio Tools - Install Free for Windows, Mac, Linux \(microsoft.com\)](https://visualstudio.microsoft.com/)



- ▶ Microsoft Visual Code
- ▶ You need a C++ SDK like MinGW
- ▶ <https://code.visualstudio.com/download>



- ▶ Eclipse CDT
- ▶ <https://eclipse.org/cdt/>





Post



Leon Bambrick

@secretGeek



There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors.

4:20 PM · Jan 1, 2010



23



1.4K



1.8K



98



[Leon Bambrick](#)

▶ Editing Code

- ▶ Naming conventions

- ▶ rules used to define name for variables, functions, classes
- ▶ help maintain code clean
- ▶ makes code easy to read
- ▶ Camel, Pascal, Kebab Case, Snake, Screaming



camelCase



kebab-case



snake_case

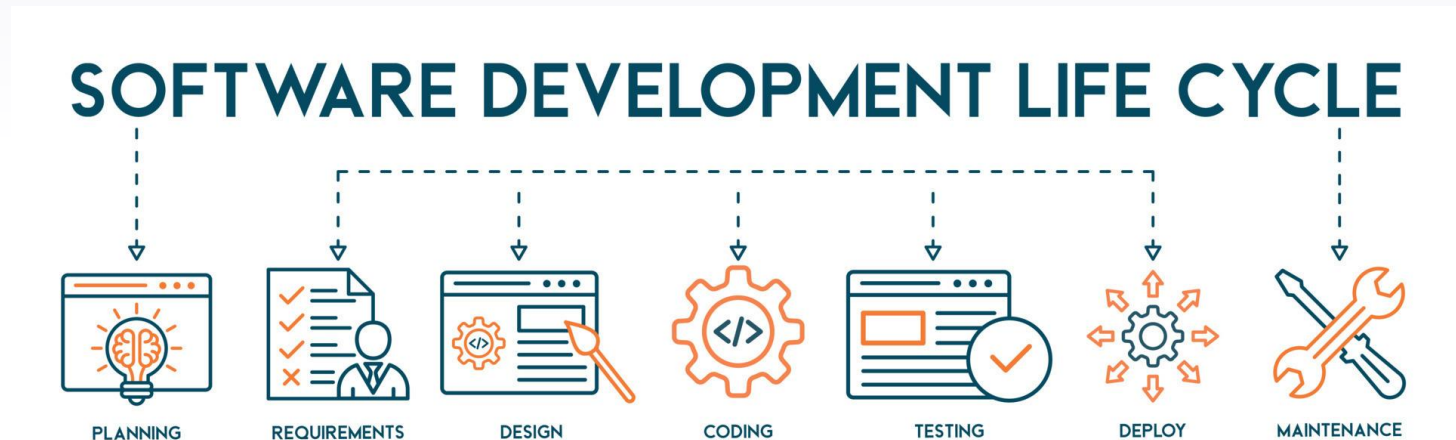
Editing Code

Case	Rule	Example
Camel	Multiple words combined. First starts with lower case, the others with upper case	<code>computeFinalGrade()</code> <code>noStudents</code>
Pascal (Upper Camel Case)	Multiple words combined. All start with upper case	<code>ComputeFinalGrade()</code> <code>NoStudents</code>
Snake	Multiple words separated by <code>_</code> (underline)	<code>compute_final_grade()</code> <code>no_students</code>
Kebab	Multiple words separated by <code>-</code> (dash)	<code>compute-final-grade()</code> <code>no-students</code>
Screaming Snake	All letters with upper case separated by <code>_</code> (underline)	<code>NO_STUDENTS</code>

Course Naming Conventions

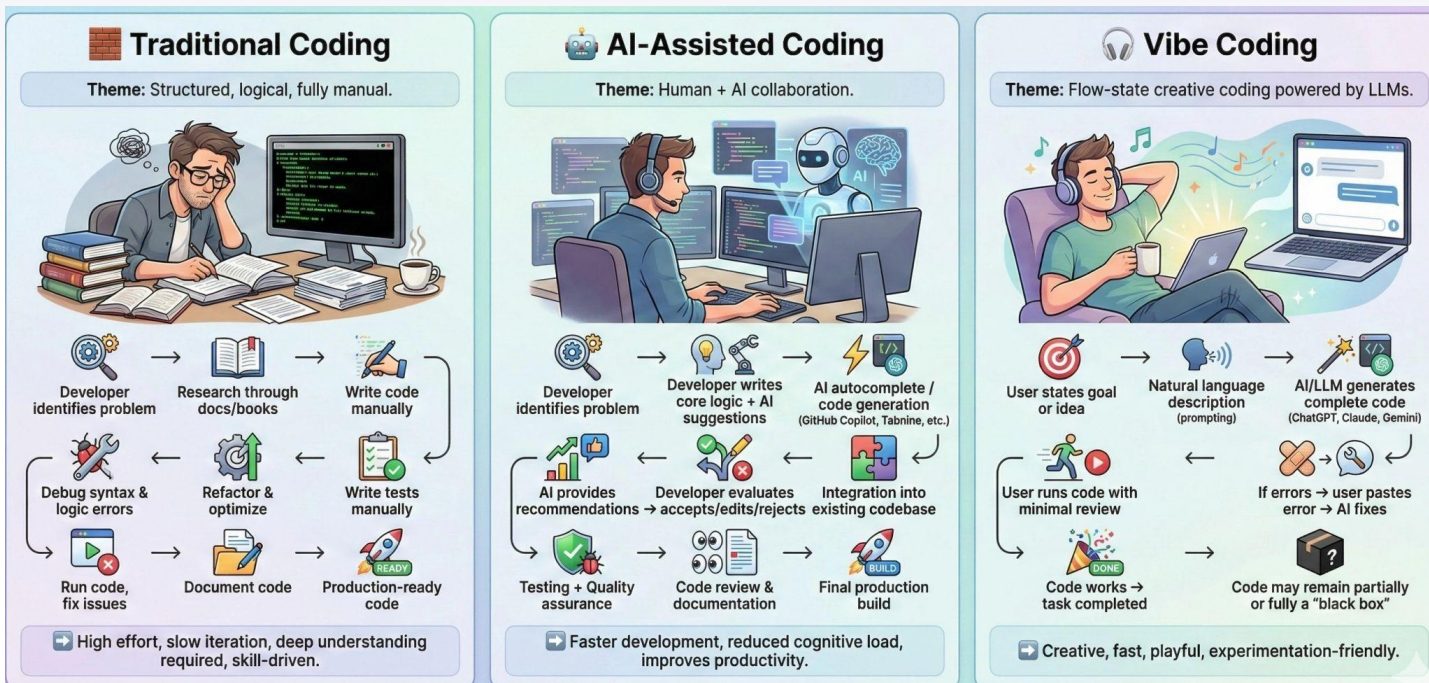
Case	Scope	Example
Camel (Lower Camel Case)	variables, functions, namespaces	<code>computeFinalGrade()</code> <code>noStudents</code>
Pascal (Upper Camel Case)	classes	<code>class Student{};</code>
Screaming Snake	static attributes	<code>NO_STUDENTS</code>

Software Development Cycle



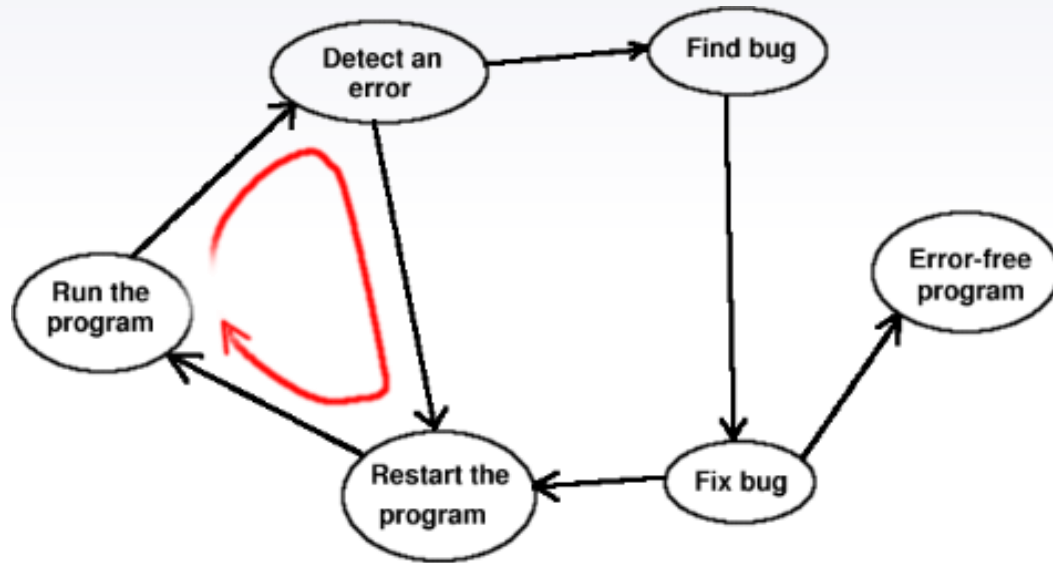
Source: Tyra Lopez

Coding in 2026



<https://www.linkedin.com/pulse/traditional-coding-vs-ai-assisted-vibe-new-spectrum-natarajan-uvrwc/>

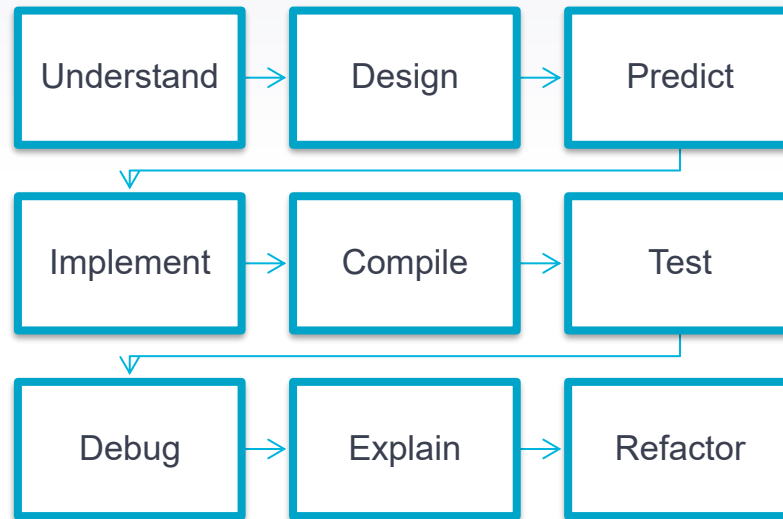
► Edit, Run, Debug



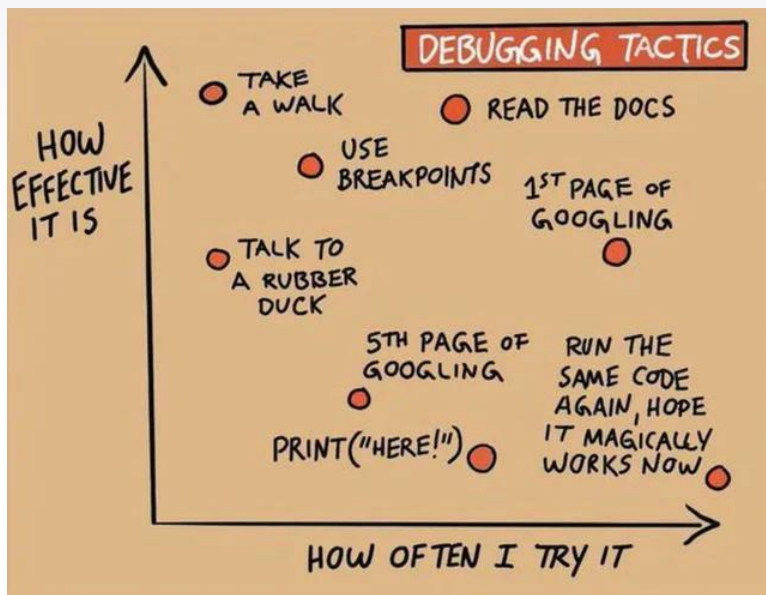
Typical Debugging Cycle

<https://lizard.sourceforge.net/explain.html>

► The full picture



Debugging



<https://programmerhumor.io/debugging-memes/debugging-tactics/>

▶ Debugging

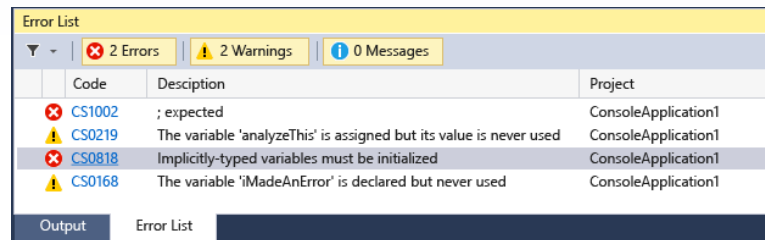
- ▶ Rubber duck debugging (or rubberducking) is a debugging technique in software engineering. A programmer explains their code, step by step, in natural language - either aloud or in writing - to reveal mistakes and misunderstandings.

[https://en.wikipedia.org/wiki/Rubber_duck_debugging]



► Compiler Errors

- ▶ Generated by code editing errors
- ▶ > 99% because syntax is not known
- ▶ Clearly highlighted by the compiler in the Errors (VS) windowand Warnings one
- ▶ **NOT related** to program complexity, algorithm used, etc.

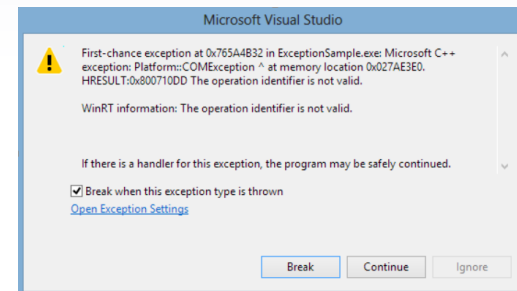


The screenshot shows the 'Error List' window in Visual Studio. At the top, there are tabs for '2 Errors', '2 Warnings', and '0 Messages'. The '2 Errors' tab is selected. Below the tabs is a table with columns for 'Code', 'Description', and 'Project'. The table contains four rows, with the first two rows highlighted in blue. The first row shows an error (CS1002) with the description '; expected'. The second row shows a warning (CS0219) with the description 'The variable 'analyzeThis' is assigned but its value is never used'. The third row shows an error (CS0818) with the description 'Implicitly-typed variables must be initialized'. The fourth row shows a warning (CS0168) with the description 'The variable 'iMadeAnError' is declared but never used'. All errors and warnings are for the project 'ConsoleApplication1'.

Code	Description	Project
CS1002	; expected	ConsoleApplication1
CS0219	The variable 'analyzeThis' is assigned but its value is never used	ConsoleApplication1
CS0818	Implicitly-typed variables must be initialized	ConsoleApplication1
CS0168	The variable 'iMadeAnError' is declared but never used	ConsoleApplication1

Run-time Errors

- ▶ Generated by mistakes in the logical implementation of the program
- ▶ > 99% because faulty or bad memory management (pointers, heap, stack, allocation, deallocation, etc.)
- ▶ Hard to identify and correctyou need to use the debugger
- ▶ They are related to the complexity of the program, the algorithm used, etc



▶ Recap C Concepts

- ▶ Data types
- ▶ Pointers (pointers to values, static and dynamic arrays, strings or arrays of chars)
- ▶ Function pointers
- ▶ References
- ▶ Functions/Methods (parameters transfers)
- ▶ Preprocessing

▶ Data types

- ▶ signed char
- ▶ unsigned char
- ▶ short
- ▶ short int
- ▶ signed short
- ▶ signed short int
- ▶ unsigned short
- ▶ unsigned short int
- ▶ int
- ▶ signed
- ▶ signed int
- ▶ unsigned
- ▶ unsigned int
- ▶ long
- ▶ long int
- ▶ signed long
- ▶ signed long int
- ▶ unsigned long
- ▶ unsigned long int
- ▶ long long
- ▶ long long int

<https://en.cppreference.com/w/cpp/language/types.html>

Pointers

- ▶ Numerical variables used to store values that represent memory addresses;
- ▶ Their size is given by the processor architecture (on 32 bits -> 4 bytes)
- ▶ definition:

```
data_type * pointer_name;
```

- ▶ init:

```
pointer_name = & variable_name;
```

- ▶ usage:

```
variable_name = * pointer_name;
```

Pointers

```
int* pi ;           // pointer to int
char** ppc ;        // pointer to pointer of char
int* ap [10];       // array of 10 pointers to int
```

- ▶ Zero value for a pointer is a **null** value. It is defined using the next macro

```
#define NULL 0
```

- ▶ or by the next constant value

```
const int NULL = 0;
```

POINTERS

Pointers arithmetic:

- ▶ For a T type pointer T^* `pointer`, the `--/++ operators` will move the current address back/forward with `sizeof(T)` bytes;
- ▶ For a T type pointer T^* `pt`, the next expression `pt + k` or `pt - k` is equivalent with moving the current address with `k * sizeof(T)` bytes;
- ▶ Adding 2 pointers has no logic and is not accepted (use 2 integers instead);
- ▶ Substraction is allowed

► Constant POINTERS

- Examples:

```
int *const p;           // constant pointer to int
int const *pint ;       // pointer to constant int
const int *pint2;        // pointer to constant int
const int *const pint2; // constant pointer to int constant
```

- Usage:

```
char* strcpy(char *p, const char *q);
```

- - prevents changing q pointer

POINTERS

Allocating dynamically memory (in HEAP):

- ▶ `new` or `new [ ]`;
- ▶ Always in Heap

Deallocating HEAP memory space:

- ▶ `delete` or `delete [ ]`;
- ▶ Only for HEAP

C++ Reference

- ▶ Represents a constant pointer that is used only to access values located at the referenced memory space
- ▶ Used to define the input arguments of a method (simpler syntax than *)

```
int vb = 10 ;  
int & refvb = vb ;  
int x = refvb ;  
refvb = 20 ;  
int & ref;  
refvb ++;  
int * pvb = & refvb;  
  
// r and i now refer to the same int  
// x = 10  
// vb = 20  
// ERROR - compiler error  
// vb = 21  
// pvb is initialized with vb address
```

► POINTER – to methods

- ▶ definition:

```
return_type (* pointer_name) (arguments list);
```

- ▶ init:

```
pointer_name = function_name;
```

- ▶ usage:

```
pointer_name (arguments list);
```

▶ POINTER – to methods

- ▶ `float (*fp)(int *);` // pointer to a method that returns a float value and receives a pointer to int
- ▶ `int *f(char*);` // **NOT a pointer** – just a method named f that returns a pointer to int
- ▶ `int* (*f p[5]) (char*);` // array of 5 pointer

New stuff in C++ (compared with C)

- ▶ Console in and out
 - ▶ Reading from console: `cin >> variable_name`
 - ▶ Printing at console: `cout << variable_name`
- ▶ Reserve space in HEAP
 - ▶ allocate: `nume_pointer = new tip_data[nr_elemente]`
 - ▶ deallocate: `delete [ ] nume_pointer`
- ▶ The **&reference** type
 - ▶ used to define methods input parameters that you want to change : `void Interschimbare( int &a, int &b)`

THANKS!

Any questions?

You can find me at:

- ▶ catalin.boja@ie.ase.ro

